

Software Design Document

For

‘SmartFarm’

Version 2.1

Prepared by

NAME	STUDENT NUMBER	REGISTRATION NUMBER
BUKOMEKO. W HANNINGTON	2300707703	23/U/07703/PS
KYEMBA CHARITY	2300700671	23/U/0671
KATONGOLE SIMON	2300709309	23/U/09309/PS
SSERUNKUMA ALVIN. C.	2300724845	23/U/24845/PS

04/05/2026

SOFTWARE DESIGN DOCUMENT: SmartFarm

1. INTRODUCTION

1.1 Purpose

This Software Design Document describes the architecture, modular design, and data structures for the **SmartFarm** management System. It serves as a technical roadmap for us the developers and stakeholders to understand how the requirements defined in the SRS will be technically implemented.

1.2 Scope

SmartFarm is a centralized web-based platform designed to digitize farm operations and unite island farm systems into one unified platform. The goal is to replace separate and manual record-keeping with an intelligent system that integrates inventory, sales, payroll, and weather monitoring. Key benefits include improved resource efficiency, data-driven decision-making, and proactive disease management.

1.3 Overview

This document organizes the system into a modular PHP/MySQL architecture. It covers high-level architectural patterns, database entity-relationship designs, and component-level logic for modules like AI Disease Detection and Weather Forecasting.

1.4 Reference Material

- SmartFarm Software Requirements Specification (SRS) v2.1, 04/05/2026.
- Open-Meteo API Documentation.

1.5 Definitions and Acronyms

- **SRS**: Software Requirements Specification
- **PDO**: PHP Data Objects (Secure database access layer)
- **API**: Application Programming Interface
- **NSSF/PAYE**: Statutory tax and social security deductions in Uganda.

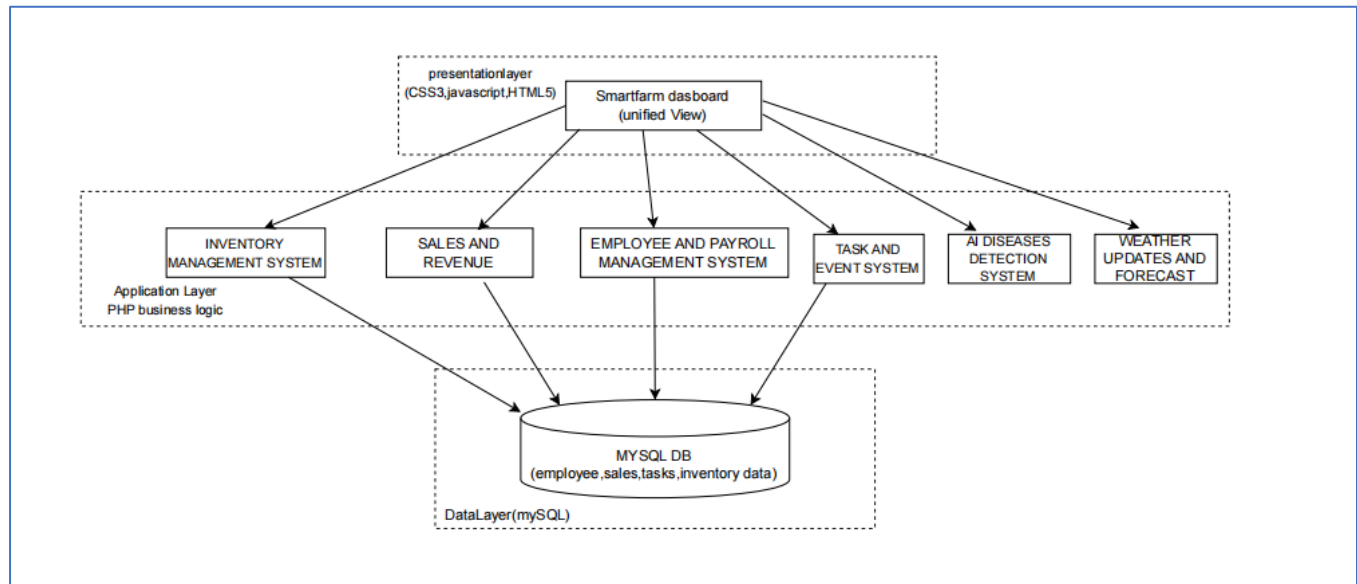
2. SYSTEM OVERVIEW

SmartFarm is a three-tier web application consisting of Client, Server, and Database layers. It provides a real-time dashboard for farm managers to monitor agricultural health, track

financial transactions, and monitor employees. The system integrates with external weather services to provide localized farming advisories.

3. SYSTEM ARCHITECTURE

A high-level architecture diagram of smartfarm intergrated system



3.1 Architectural Design

The system follows a Modular Client-Server Architecture.

- **Presentation Layer (Frontend):** HTML5, CSS3, and JavaScript handle the user interface and responsive dashboard.
- **Application Layer (Backend):** PHP modules process business logic, such as payroll calculations and inventory thresholds.
- **Data Layer (Storage):** A MySQL database stores persistent records for employees, sales, and tasks.

Subsystem Collaboration:

The Dashboard acts as the primary player, pulling data from the Inventory, Sales, and Weather modules to present a unified "Farm Status" view.

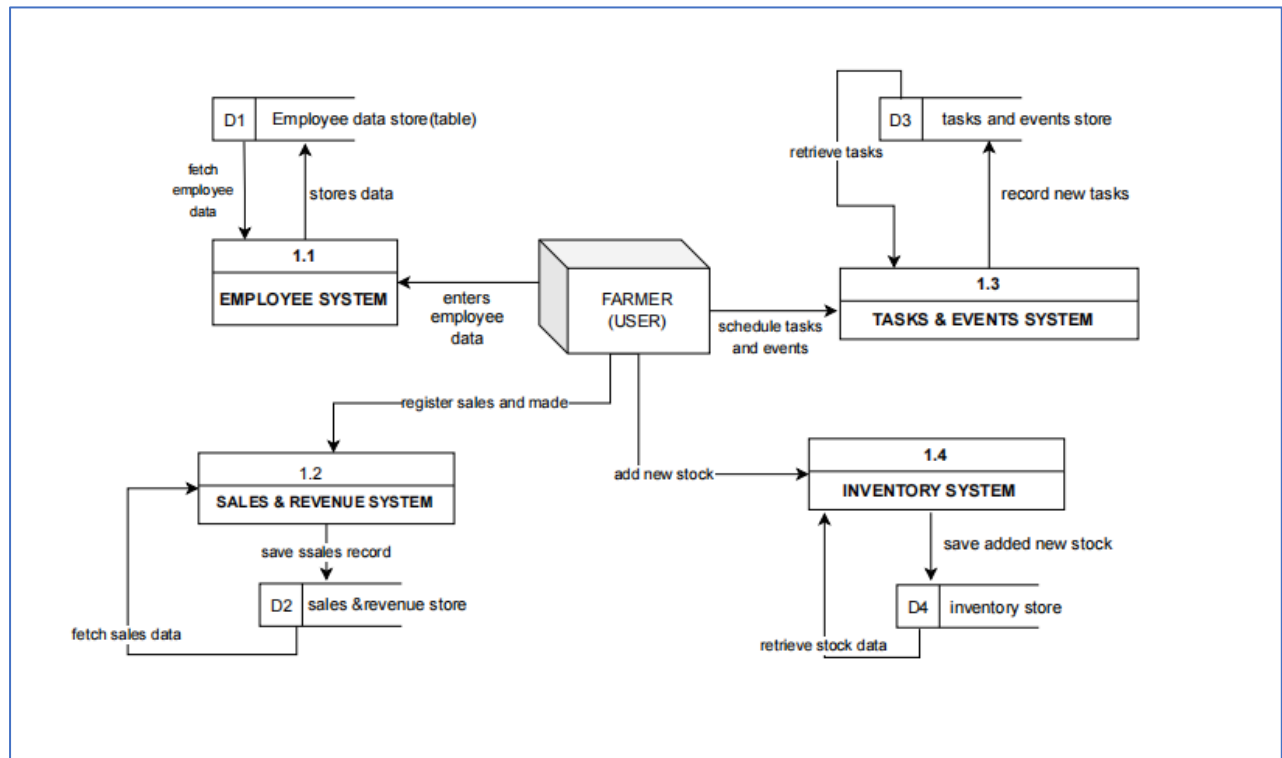
3.2 Decomposition Description (Functional)

The system is decomposed into the following functional modules:

1. **Inventory Controller:** Manages stock levels and triggers "Shortage Alerts."
2. **Financial Engine:** Processes sales data and calculates total revenue.

3. **Human Resource Module:** Handles employee records and statutory payroll deductions.
4. **Intelligence Module:** Interfaces with Open-Meteo API and runs the symptom-based AI diagnosis logic.

Level 1 data decomposition diagram for smartfarm intergrated system



3.3 Design Rationale

A PHP/MySQL stack was selected for its high availability on local servers and compatibility with low-resource environments typical in agricultural sector. A modular approach was chosen to minimize latency and simplify deployment for local farm servers.

4. DATA DESIGN

4.1 Data Description

Information is transformed into a Relational Database Management System (RDBMS). Data is stored in normalized tables to ensure referential integrity (e.g., a Task cannot exist without an assigned Employee).

4.2 Data Dictionary

Entity	Attributes	Description
--------	------------	-------------

User	ID, Username, Password, Role	System access credentials.
Employee	ID, Name, Role, Salary, Status	Personnel and payroll tracking.
Inventory	Item_ID, Qty, Min_Threshold, Cost	Resource and stock management.
Sale	Sale_ID, Product, Qty, Price, Status	Revenue and transaction records.
Task	Task_ID, Title, Due_Date, Priority	Operational scheduling data.

5. COMPONENT DESIGN

Inventory Alert Algorithm (Pseudocode)

```

FUNCTION checkStockLevels():
  FOR EACH item IN InventoryTable:
    IF item.quantity < item.min_threshold:
      CREATE alert_notification
      SET dashboard_status = "Attention Required"
  RETURN alerts

```

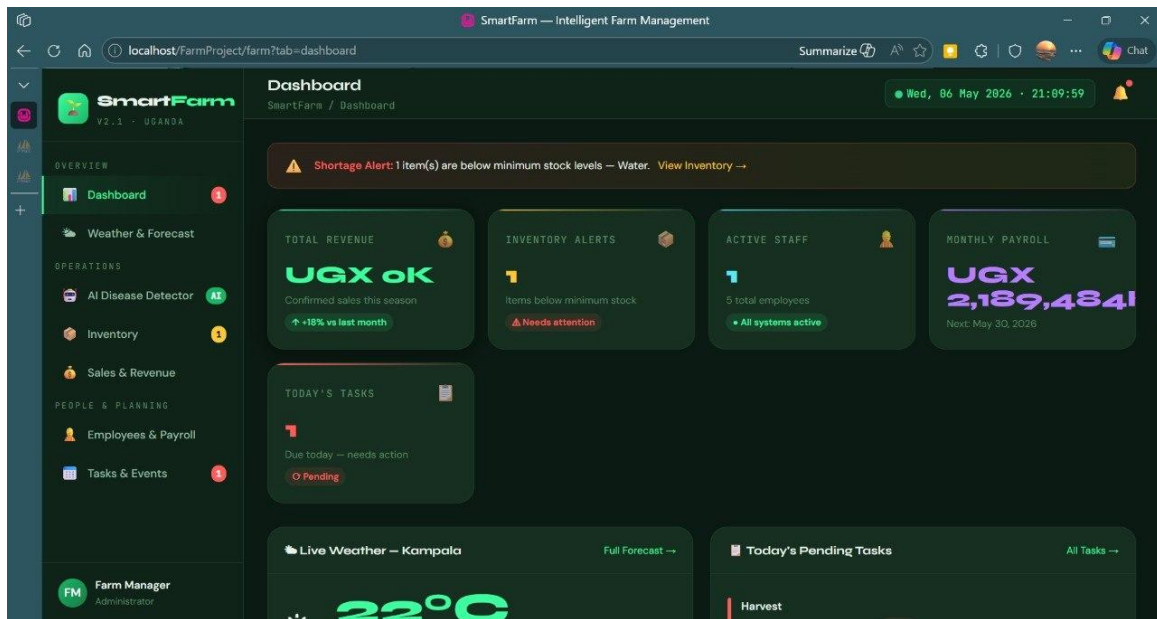
The pseudo code above enforces the functionality of alerting the user of the stock shortages by displaying a warning message on the dashboard.

6. HUMAN INTERFACE DESIGN

6.1 Overview

The user interface is a Sidebar-driven Dashboard. Users navigate between "Operations" (Sales/Inventory) and "Planning" (Tasks/Events). Real-time feedback is provided via color-coded alerts.

6.2 Screen Images



6.3 Screen Objects and Actions

- **Quick-Action Buttons:** Allow recording of sales.
- **Status Toggles:** Change task status from "Pending" to "Completed."
- **Symptom Tags:** Interactive buttons in the AI module to select disease symptoms.

7. REQUIREMENTS MATRIX

System Component	SRS Requirement Traceability
Dashboard UI	FR-1, FR-2, FR-3, FR-4, FR-5
Weather API Handler	FR-6, FR-7, FR-8, FR-9
AI Diagnosis Logic	FR-10, FR-11, FR-12
Inventory DB system	FR-14, FR-15, FR-16, FR-17
Payroll Processor	FR-24, FR-25, FR-26